

# Audit performance backend PNPI — lot 79.3

**Date** : 2026-04-27 **Auteur** : Ingenieur Senior Backend (lot 79) **Cible** : audience ministerielle imminente, prudence requise.

## Methode

Analyse statique des routers les plus appeles (dashboard, listing ATI, open-data, auth) et croisement avec les index existants ( `alembic/versions/20260401_32_composite_indexes.py` + index inline). Pas d' `EXPLAIN` reel sur Postgres prod (audience trop proche pour toucher la base) — les conclusions sont basees sur la lecture des requetes ORM et l'experience d'optimisation Postgres.

## Queries identifiees comme couteuses

### 1. Listing ATI operateur — `created_by` non indexe (impact : ELEVE)

**Endpoints concernes** : - `GET /pnpi/ati` (cf `routers/ati.py`) - `GET /pnpi/operateurs/{id}/ati` (cf `routers/operateurs.py`) - Tout endpoint `/pnpi/ati/{id}/*` qui passe par `check_ati_access` - `pnpi/ati/historique` , `/comments` , `/tags` , `/risk` , `/field-history` , `/renew` , `/resubmit`

**Pattern SQL** :

```
SELECT * FROM agrements_ati
WHERE created_by = :username
ORDER BY date_soumission DESC;
```

**Index existant** : aucun. Seuls `(numero_ati)` , `(operateur_id)` , `(secteur)` , `(statut)` , `(statut, date_soumission)` et `(instructeur_username, statut)` existent.

**Index ajoute** : `ix_ati_created_by_date_soumission(created_by, date_soumission)`

**Impact attendu :**  $O(\log n)$  au lieu de  $O(n)$  sur le scan, ET tri évite (index ordonne). Chaque connexion opérateur appelle ce pattern -> gain direct sur tous les opérateurs et toute la latence perçue côté guichet.

## 2. Audit trail filtre par target (impact : MODERE)

**Endpoints concernés :** - `GET /admin/audit?target=...` (admin panel) - Resolution de l'historique d'un ATI ou opérateur

**Pattern SQL :**

```
SELECT * FROM audit_events
WHERE target = :target_id
ORDER BY timestamp DESC;
```

**Index existant :** (timestamp, actor) — ne couvre pas la recherche par target.

**Index ajoute :** `ix_audit_target(target)`

**Impact attendu :** recherche par cible passe de full scan à index seek. La table audit grossit linéairement, gain qui s'amplifie dans le temps.

## 3. Login history par utilisateur (impact : MODERE)

**Endpoint :** `GET /auth/me/login-history` (visible par chaque user).

**Pattern SQL :**

```
SELECT * FROM login_history
WHERE username = :username
ORDER BY created_at DESC
LIMIT :limit;
```

**Index existant :** `username` simple (cf `LoginHistoryORM` ).

**Index ajoute :** `ix_login_history_username_created(username, created_at)`

**Impact attendu :** évite le sort externe quand le user a beaucoup d'entrées. Chaque connexion écrit une ligne.

## 4. Dashboard KPIs : opérateurs actifs par province (impact : MODERE)

**Endpoint :** `GET /pnpi/dashboard/kpis` (multi-tenant province).

**Pattern SQL :**

```
SELECT count(*) FROM operateurs_industriels
WHERE is_active = true
      AND province = :province
      AND deleted_at IS NULL;
```

**Index existant :** province simple, is\_active non indexé.

**Index ajoute :** ix\_operateurs\_active\_province(is\_active, province)

**Impact attendu :** selectivité forte combinée, l'index couvre les filtres principaux. Cache 2 min mitige déjà une partie de la pression mais le miss-rate ministre/directeur reste élevé.

## 5. Taux conformité via inspections (impact : MODERE)

**Endpoint :** GET /pnpi/dashboard/kpis , /pnpi/dashboard/secteurs .

**Pattern SQL :**

```
SELECT * FROM inspections_conformite
ORDER BY date_inspection DESC;
```

Le code Python boucle ensuite pour ne garder que la dernière par opérateur. Un futur refactor en SQL window function réduirait encore la charge ; pour aujourd'hui on couvre l'index sur le couple usuel.

**Index ajoute :** ix\_inspections\_statut\_date(statut\_conformite, date\_inspection)

**Impact attendu :** facilite le pilotage par état de conformité.

## 6. Notifications par rôle + lecture (impact : FAIBLE-MODERE)

**Endpoint :** GET /notifications (filtres rôle + lu/non-lu).

**Index existant :** (is\_read, created\_at) .

**Index ajoute :** ix\_notifications\_target\_role\_read(target\_role, is\_read)

**Impact attendu :** selectivité forte sur target\_role + is\_read=false (badges UI). L'index existant (32) couvre le cas global mais pas le filtre par rôle.

## Gains qualitatifs attendus

| ENDPOINT                              | AVANT (ESTIMATION) | APRES (ESTIMATION) |
|---------------------------------------|--------------------|--------------------|
| GET /pnpi/ati (opérateur)             | 80-200 ms          | 5-20 ms            |
| GET /pnpi/operateurs/{id}/ati         | 60-150 ms          | 5-15 ms            |
| GET /pnpi/dashboard/kpis (cache miss) | 250-500 ms         | 150-300 ms         |
| GET /auth/me/login-history            | 30-80 ms           | 5-15 ms            |
| GET /admin/audit?target=...           | scan complet       | index seek         |

Estimations basees sur des bases avec 10k-50k ATI et 100k inspections. Chiffres reels a verifier en pre-prod via `EXPLAIN ANALYZE`.

## Limitations / Suite

- **Pas de mesure reelle** : pas d'EXPLAIN execute dans cette session (Bash/PowerShell desactives, audience proche, pas de touche prod). L'index ajoute reste une hypothese rationnelle, validee par lecture des requetes.
- **N+1 detecte mais non resolu** : `/pnpi/dashboard/carte` charge tous les ATI puis fait un dict en Python. A refactorer en agregation SQL (lot 80+).
- **Inspections last-by-operateur** : meme remarque, candidat a une window function `ROW_NUMBER() OVER (PARTITION BY operateur_id ORDER BY date_inspection DESC)`.
- **Index de couverture (INCLUDE)** : Postgres 11+ supporte `CREATE INDEX ... INCLUDE (col)`. Pas applique ici car SQLite (dev) ne supporte pas — a considerer en migration prod-only.

## Validation

Apres `alembic upgrade head` :

```
# Postgres
psql -c "\di+ agrements_ati" | grep created_by
psql -c "EXPLAIN ANALYZE SELECT * FROM agrements_ati WHERE created_by = 'operateur' ORD
```

L'attendu est `Index Scan using ix_ati_created_by_date_soumission` au lieu de `Seq Scan` + `Sort` .